

Mohammed Aljomaily

Portfolio

Computer Engineering Student | Embedded Systems, PCB Design & RF

Beylikdüzü, Istanbul, Turkey | +90 535 273 6199 | mohammedaljomaily06@gmail.com

v01d.dev | github.com/V01Ddev

Preface

My name is Mohammed Aljomaily. I am a Computer Engineering student at Istanbul Aydın University, and I am applying for an internship at Baykar.

I came into engineering through software, somewhat by accident. During a gap year I took an internship at Applab in Doha, and it turned into fourteen months of real work: I built backend services for an AI video platform used by Al Jazeera, and later an AI chatbot for the General Authority of Customs. It was serious work and I am proud of it. But it was never the part of engineering I went looking for. The thing I actually did with my own time was different. I was designing circuit boards, etching them at my kitchen table, soldering them, and flying what I had built.

That is the work I want to do properly.

Over the past year I have designed and hand-etched my own PCBs at home, because paying for fabrication and shipping to Turkey did not make sense for the size of the boards I was making. I designed a two-layer ESP32 board carrying two nRF24L01 transceivers on separate SPI buses, and discovered after the first layout that two layers are close to impossible to produce by hand. I widened the vias, reworked the routing, and made it again. The board works. I have built and tuned a 5-inch FPV quadcopter and written the firmware for the radio link on my fixed-wing designs. I published a KiCad footprint library for the SSD1306 OLED because I needed one that worked in a current version of KiCad and could not find it, and I use it in my own boards. I keep a written preflight checklist for my aircraft, because I would rather catch a loose propeller on the bench than in the air.

Baykar is one of a small number of places where the thing I do for my own interest is the thing the company does at scale. I want to take problems and make real-world solutions that combine both hardware and software.

I am not arriving with only coursework behind me. I can take a design from a schematic to a board I can hold in my hand, and I have done that several times. I work in C and C++ on ESP32 and ESP8266, in KiCad for boards and FreeCAD for mechanical parts, and I am comfortable at a bench with a soldering iron and a multimeter. I also know how to write software that keeps running after the demo, which I learned the hard way on systems that real people depended on.

The rest of this document is a record of what I have designed, built and flown. Almost everything here was made at a desk at home, including the circuit boards themselves. Where a project is unfinished I have said so.

5-Inch FPV Quadcopter

2026

A complete 5-inch FPV aircraft, built from parts and configured from scratch: **Mario 5** frame, **Mamba MK4 F722** flight controller, **F40 V2** motors, and **DJI O4 Lite** video on **6S**. I configured Betaflight, set up the **ExpressLRS 2.4 GHz** control link between a RadioMaster Pocket running EdgeTX and the receiver, and tuned the aircraft to fly the way I wanted it to rather than the way it arrived.

I wrote the preflight checklist for this aircraft myself, as a ground-check procedure I run before every session. It is not a formality: on a machine with propellers, batteries and a radio link, the failures that hurt are the boring ones.

Alongside the airframe I built out the rest of the workflow, including GyroFlow stabilisation for footage from a GoPro HERO7 with ND filters, and a custom EdgeTX sound pack and 3D-printed stick ends for the radio.



The finished 5-inch build.

Dual-Transceiver ESP32 Jammer Board

2026

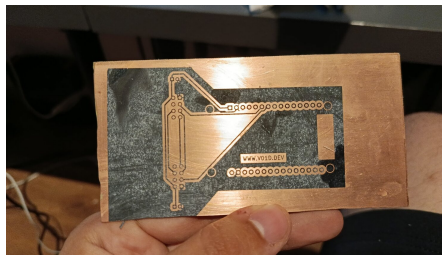
A two-layer ESP32 jammer board I designed in **KiCad** for educational study of 2.4GHz protocols, and then fabricated by hand at home: an **ESP32 DevKit V1** carrying **two nRF24L01+PA+LNA** transceivers, each on its own SPI bus (HSPI and VSPI) so both radios operate independently, plus an SSD1306 OLED for status and an optional TP4056 charging circuit for a Li-Ion cell. The finished board is 64.14×77.47 mm.

I did not use a fabrication house. Shipping to Turkey plus the cost of a local service made no sense for boards this size, so I generate the artwork, transfer it, etch it, drill it and assemble it myself. That decision shaped the design: the first layout was two layers and turned out to be close to impossible to produce by hand, so I enlarged the vias, reworked the routing for the process, cleared every DRC error, and made it a second time. The second board works.

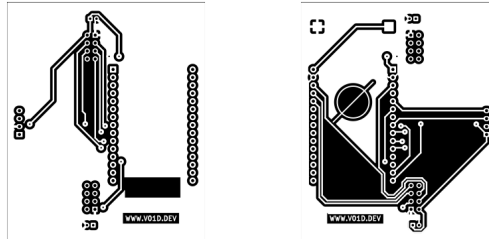
The firmware is adapted from an existing open-source project rather than written from scratch. I wanted my hardware to be fully compatible with it, so the engineering work on my side was the board, the schematic capture, the custom footprints and the fabrication.



The fabricated board. Bare etched substrate, no solder mask, assembled by hand.



Toner transferred onto the copper, before etching.



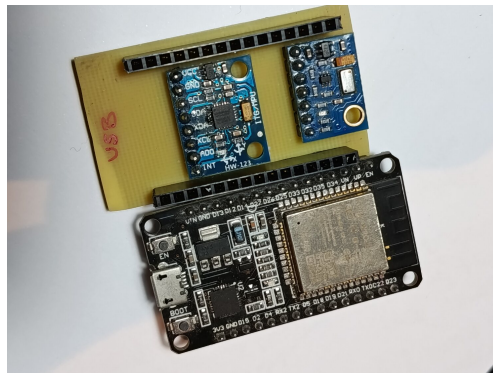
Front and back copper layers, exported from KiCad. These are the etch artwork.

Flight Telemetry & Attitude Indicator

2026

An ESP32 telemetry system for small aircraft, pairing an **MS5611** barometer with an **MPU6050** inertial measurement unit. I fused the two sensors through a complementary filter (to correct the gyroscope's drift against the accelerometer's noise) and streamed attitude and altitude over serial at roughly 50 Hz.

The ground station is written in Python with **PySide6** and **Qt QML**, drawing a live artificial horizon from the incoming telemetry. I started it in Pygame and migrated it to QML partway through, because the original approach could not render smoothly enough to be useful as an instrument.



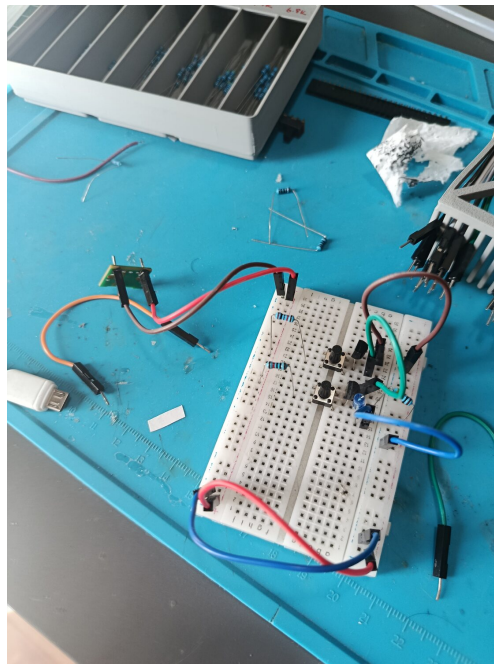
The telemetry board.

Discrete Transistor Logic Gates

2026

A board that implements **AND**, **NOT**, **OR** and **NAND** gates from individual transistors, with no logic ICs anywhere: seven **PN2222A** NPN transistors in resistor-transistor logic, with LEDs as outputs and tactile switches as inputs.

I built this to work below the abstraction layer. It is easy to use a 74-series chip and never think about what is inside it. Designing the same gates from transistors means reasoning about base current, saturation and voltage levels directly. I designed the schematic, routed the two-layer board with a ground plane, and fabricated the first revision. It is currently under debug, and I am working through the fault.



The gate circuits on a breadboard, before committing to the PCB.

RC Aircraft & Radio Link

2025–2026

Two fixed-wing designs, drawn and built from first principles rather than from a kit. **FCF-1** uses a 9 mm carbon fibre main spar with a **NACA 63412** airfoil, and I modelled the airframe in **FreeCAD**: wing structure, servo mounts, pushrods, control horns, aileron hinges and motor mount, all as parts that could actually be cut and printed. It has flown. A later design, **Simple-Glider**, was built to be easy to reproduce from foam board and printed parts, with a full build guide covering dimensions and part counts.

For the radio link on these aircraft I wrote the transmitter and receiver firmware myself: four analog channels encoded over **ESP-NOW** at 20 Hz, driving servos on the receiving end, with a failsafe that recentres every channel if the link is lost for three seconds.

Published KiCad Library

2026

The SSD1306 0.96-inch OLED module is one of the most common parts in hobby electronics, and I could not find a KiCad symbol and footprint for it that worked in a current version of KiCad. So I made one and published it: a native **KiCad 9** symbol and footprint, converted from an earlier library with the original **CC BY-NC-SA** attribution preserved as its licence requires.

It is not a file sitting in a drawer. I use this footprint in my own boards, including the RF board above.

Home PCB Fabrication

ongoing

Everything with a circuit board in this portfolio was made by hand, and the process itself has been a project. I generate artwork from KiCad, transfer it to copper, etch, drill and assemble, and I designed for that process on the boards above. I modelled a **toner transfer tool** in FreeCAD specifically for this workflow, and I write up the method on my site because the information that helped me most came from other people doing the same thing.

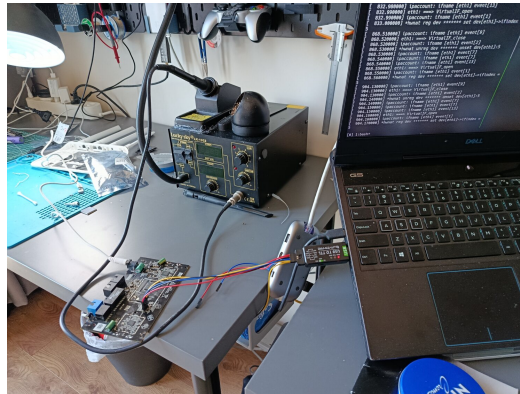
I also model and print mechanical parts, from enclosures for the boards above to a reversible USB power modification for a FlySky i6X transmitter that fits into the battery compartment without altering the case.

Router Firmware Reverse Engineering

2026

I took a Xiaomi router apart to understand how it boots. I located the UART header on the board and

connected to it to capture the boot log, which is the most direct way to see what a device does before any of its own software takes over. The console turned out to be locked, so I moved to the flash instead and extracted the firmware image, which showed the firmware was built on an open-source base. I stopped there when the university term started.



Capturing the boot log over UART.

Software Work

2025–2026

My professional experience is in software, and it is where I learned what production actually means.

At **Applab** in Doha I was the primary author of the backend for **Al Jazeera's** AI video editing and translation platform: a Quart API, an asynchronous job worker, and the queueing between them on Azure Container Apps, Cosmos DB and Storage Queues. I integrated Azure Cognitive Services and Azure OpenAI for transcription and translation across 14 locales, and built the failure handling (poison-message routing, bounded retry, per-action logging) that a long-running pipeline needs. Separately I built an AI chatbot for the **General Authority of Customs** on FastAPI, Semantic Kernel and Azure AI Search.

Outside work, I built a retrieval-augmented chatbot over university documentation using LangChain and MongoDB Atlas Vector Search, with an incremental ingest pipeline that hashes each page so only what changed gets reprocessed. I also maintain **TIPOT**, a PDF organising tool that has received and merged a pull request from an outside contributor.

Toolchain. C and C++ on ESP32/ESP8266, Python. KiCad 9/10 for schematic capture and PCB layout, including custom footprint authoring. FreeCAD for mechanical design, FDM printing for parts. SPI, I²C, UART, PWM and ADC at the hardware level; nRF24L01, ESP-NOW, ExpressLRS, Bluetooth and IR for radio. Bench work with a soldering iron, multimeter and serial telemetry.